

1. OBJETIVO E ETAPAS

O objetivo do projeto é desenvolver um compilador para a linguagem de programação Java Script Simplificado (JSS) descrita nesse especificação. O projeto pode ser desenvolvido individualmente ou em equipes de no máximo três membros.

O projeto deve ser desenvolvido em duas etapas valendo os seguintes pontos:

- 1) Etapa de análise (*front-end*) - 6,0 pontos;
- 2) Etapa de síntese (*back-end*) - 4,0 pontos.

* Membro de equipe ou equipe que não apresentar no dia agendado será penalizado com metade da nota obtida na apresentação.

2. DATAS E PONTUAÇÃO

- 18/junho (5ª feira): apresentação prévia da 1ª etapa (*front-end*) individualmente para professor/monitor, valendo 2,0 pontos.
- 25/junho (5ª feira): apresentação da versão final da 2ª etapa (*front-end*) individualmente para professor/monitor, valendo 4,0 pontos e submissão no Sigaa até 23:59.
- 2/julho (5ª feira): apresentação da 2ª etapa individualmente para professor/monitor, valendo 4,0 pontos e submissão no Sigaa até 23:59.

Todas as terças-feiras de 16 a 30 de junho haverá atendimento a dúvidas específicas por grupo de trabalho com presença facultativa.

3. DESENVOLVIMENTO

3.1. *Front-end*

Será avaliado os seguintes itens na implementação do front-end:

- a) Regras léxica e sintática.
- b) Regras semântica.
- c) Execução do compilador em linha de comando da seguinte forma:
 - I. ler da entrada padrão arquivo com programa escrito na linguagem;
 - II. gerar na saída padrão uma confirmação ou erro lexo/sintático/semântico indicando a linha onde ocorreu.
- d) Manual (arquivo *readme*) contendo informação do *front-end* sobre
 - I. Instalação de bibliotecas pré-requisitos para execução
 - II. execução com exemplos de códigos com erro e sucesso

A apresentação da versão prévia do front-end deve constar pelo menos implementações dos itens a) e c) valendo 1,0 ponto para cada um desses itens sendo os demais itens facultativos. Os problemas indicados pelo professor ou monitor devem ser corrigidos para a apresentação da versão final.

A apresentação da versão final deve constar todos os itens da avaliação valendo 1,0 ponto cada. O *front-end* final deve ser submetido no SIGAA com todos os itens requisitados corrigidos comprimidos no formato ZIP.

3.2. *Back-end*

Apresentar ao professor a implementação dos seguintes itens, valendo 1,0 ponto cada:

- a) Regras para tradução da parse tree ou árvore sintática no passo anterior para uma linguagem intermediária, podendo ser LLVM ou JASMIN.
- b) Geração de executável em seu sistema operacional a partir da linguagem intermediária.
- c) Manual (arquivo *readme*) contendo informação do *back-end* e o *front-end* da etapa anterior (compilador completo) sobre
 - I. Instalação de bibliotecas pré-requisitos para execução
 - II. execução com exemplos de códigos com erro e sucesso

Após a apresentação, os problemas indicados pelo professor ou monitor devem ser corrigidos, e o *back-end* mais o *front-end* anteriormente desenvolvido (compilador completo), deve ser submetido no SIGAA com todos os itens requisitados comprimidos no formato ZIP.

4. ESPECIFICAÇÃO

Java Script Simplificado (JSS) é uma linguagem baseada na sintaxe do JavaScript, porém projetada com tipagem forte e estrutura simplificada para ser compilada. Essa linguagem é sensível (case-sensitive) em relação a letras maiúsculas e minúsculas nos nomes das variáveis, funções e palavras reservadas. (i.e., “var”, “VAR” e “VaR” representam identificadores diferentes).

A linguagem JSS deve suportar declarações e chamadas de funções, incluindo chamadas recursivas. A linguagem possui funções built-in, utilizadas para realizar entrada e saída de dados via console do sistema operacional, que serão as únicas formas de se comunicar com o mundo exterior.

Nenhuma característica de outra linguagem deve ser reconhecida pelo compilador.

4.1 Regras para uso de variáveis, constantes e seu escopo

Variáveis podem ter identificadores de qualquer tamanho maior que zero e devem obrigatoriamente começar com uma letra ou underscore. Após isso, podem conter qualquer letra, dígito ou o símbolo underscore (_). Os identificadores não podem ter o mesmo nome de uma palavra reservada da linguagem (*if*, *while*, etc.) nas sentenças descritas nas próximas seções.

O escopo define a visibilidade de um identificador (variável ou constante) onde pode ser acessado.

- Escopo Global: Identificadores declarados fora de qualquer função ou bloco ({ ... }) são considerados globais e acessíveis de qualquer parte do programa.
- Escopo de Bloco (Local): *let* e *const* possuem escopo de bloco. Isso significa que são visíveis apenas dentro do bloco ({ ... }) em que foram declaradas (incluindo blocos de funções, *if*, *while*, etc.) e em sub-blocos.
- Re-declaração: Não é permitido declarar novamente um identificador usando *let* ou *const* no mesmo escopo.

4.2 Tipos de variáveis

A linguagem JSS possui tipos os primitivos e tipos derivados a seguir.

4.2.1 Tipos Primitivos

- inteiros (int): Números inteiros (ex: 1, -5). Valor padrão: 0;
- reais (real): Números de ponto flutuante (ex: 1.5, -10.8E2). Valor padrão: 0.0;
- strings (str): Sequências de texto (ex: "Olá\n"). Valor padrão: string vazia;
- booleanos (bool): Valores lógicos (*true* ou *false*). Valor padrão: *false*;

4.2.2 Tipos Derivados

- Vetor: uma coleção ordenada de valores do mesmo tipo, acessada através de um índice numérico. Apenas na declaração é possível atribuir uma lista de valores a um vetor. Vetores constantes não podem ter seus valores alterados. Quando não iniciado, cada elemento tem o valor padrão do tipo.

Sintaxe:

```
<identificador>[dimensão] = [<lista_de_expressões>]
<identificador> [<expressão>]
```

Exemplo:

```
let int [3] l1 = [0+1, 1+2, 2+3];
let int [3] l2;
l2 [0] = 1;
l2 [1] = 10;
l2 [2] = 100;
const str[3] nomes = ["Ana", "Bruno", "Carlos"];
```

- Classes: definição de tipos para declarar objetos que possuem um nome, atributos e um ou mais métodos, sendo que um deles deve ser constructor. Os atributos devem ser declarados antes de qualquer método e para criar um objeto de uma classe é preciso usar o a palavra reservada *new* seguida do nome da classe com os argumentos necessários para constructor.

Sintaxe de classe:

```
class <nome_da_classe> {
    (<tipo> <identificador_do_atributo>;)*
    <nome_da_classe> constructor (<lista_de_parâmetros>) {
        (this.<identificador_do_atributo> = <expressão>)*
    }
    (<tipo> <nome_do_método>(<lista_de_parâmetros>)) {
        ...
        return <expressão>
    }*
}
```

Exemplo:

```
class Ponto {
```

```

    int x;
    int y;

    Ponto constructor (int x, int y) {
        this.x = x;
        this.y = y;
    }

    int soma () {
        return this.x + this.y
    }
}

```

O objeto de uma classe é uma coleção de atributos e métodos, cujo valor padrão é *null* e um objeto constante não pode ter seus atributos alterados.

Sintaxe de objeto:

```

new <nome_da_classe>(<lista_de_argumentos>)
<identificador>.<atributo>

```

Exemplo de objeto:

```

let Ponto p1;
p1 = new Ponto (1 ,2);
p1 = null;
const Ponto p2 = new Ponto (10 ,100);

```

4.3 Operadores

A linguagem deve suportar os operadores aritméticos, lógicos e de atribuição unário ou binário abaixo:

- ! negação, inverte o valor booleano ao qual foi aplicada – unário;
- + mais unário, mantém o valor inteiro ou real ao qual foi aplicada – unário;
- - menos unário, inverte o valor inteiro ou real ao qual foi aplicada – unário;
- + soma ou concatenação de strings – binário;
- - subtração – binário;
- multiplicação – binário;
- / divisão inteira e real – binário;
- % resto de uma divisão de inteiros – binário;
- ** exponenciação de inteiros – binário;
- == igual – binário;
- != diferente – binário;
- > maior que – binário;
- >= maior ou igual – binário;
- < menor que – binário;
- <= menor ou igual – binário;
- && E lógico (conjunção) – binário;
- || OU lógico (disjunção) – binário;
- ++ incremento de uma unidade (pré-fixado) – unário;
- -- decremento de uma unidade (pré-fixado) – unário;
- = atribuição, pode ser simples ou composta realizando uma operação simultaneamente (exemplo: +=, -=) – binário;

A precedência de operadores segue os seguintes critérios

- I. Quando dois operadores diferentes forem utilizados ao mesmo tempo, o operador de maior precedência será avaliado primeiro;
- II. Operadores de mesma precedência serão avaliados da esquerda para a direita, exceto atribuições e exponenciações;
- III. Operadores aritméticos binários fazem conversão implícita para real caso um dos operandos seja real, assim como o operador de concatenação faz para string quando um dos operandos é string;
- IV. É possível forçar a precedência de uma operação, colocando-a entre parênteses;
- V. Os operadores lógicos binários operam em curto-circuito, ou seja, o segundo operando não será avaliado se o resultado da expressão puder ser determinado apenas pelo primeiro.

Tabela 1: Precedência de operadores (1 é a mais alta).

Operador	Operador	Operador	Operador
!	1	booleano	booleano
+ e - (unários)	1	inteiro ou real	inteiro ou real
++, --	1	inteiro ou real	inteiro ou real
**	2	inteiro	inteiro
* e /	3	inteiro e/ou real	inteiro e/ou real
%	3	inteiro	inteiro
+ e -	4	inteiro e real	inteiro e real
+	4	string e algum tipo simples	string
>, >=, <= e <	5	todos, exceto vetor	booleano
== e !=	5	booleano	booleano
&&	6	booleano	booleano
	7	todos	booleano
= (e compostos)	8	booleano	(tipo do operando)

4.4 Funções

Funções são declaradas com a palavra chave *function* e têm um nome, uma lista de parâmetros e um tipo de retorno, que não pode ser vetor. No caso em que não há retorno a função deve ser declarada com a palavra chave *void* e não precisam obrigatoriamente de um retorno com o comando *return*.

Funções tem escopo global (não pode haver declaração de funções aninhadas) e não deve ter o mesmo nome que outra função, variável ou constante já declarada. Adicionalmente, funções podem chamar outras funções ou ser chamadas recursivamente.

Sintaxe de declaração (função):

```
function <tipo> <nome_da_função> ( <lista_de_parâmetros> ) {  
    ...  
    return <expressão>;  
}
```

Exemplo:

```
function real media3(real nota1, real nota2, int nota3) {  
    let real media = (n1+n2+n3)/3;  
    return media;  
}
```

4.5 Funções Nativas

O compilador deverá reconhecer previamente um conjunto de funções nativas.

- Função input

Requisita a entrada de números inteiros, reais e strings a partir do teclado. O valor fornecido é armazenado na variável usada na chamada do comando input. O comando de leitura interrompe a execução do programa até que o usuário digite um valor (o valor é confirmado após pressionar a tecla "<Enter>").

Sintaxe de chamada:

```
input(<lista de variáveis>);
```

- Função console.log

envia saída para o console e a cada chamada também é enviada uma quebra de linha, exceto entre os itens de print (lista de expressões). A lista de expressões deve ser separada por vírgulas. Os argumentos são separados por um espaço na impressão.

Sintaxe de chamada:

```
console.log(<lista_de_expressões>);
```

- Funções de Conversão (Casting)

Permitem a conversão explícita entre os tipos primitivos suportados (str, int, real, bool). Qualquer tipo primitivo pode ser convertido para string, ao passo que numéricos e booleanos (int, real e bool) são intercambiáveis.

Sintaxe de chamada:

```
int(<expr>): Converte para inteiro. Reais são truncados. Booleans: true → 1, false → 0.  
real(<expr>): Converte para ponto flutuante. Booleans: true → 1.0, false → 0.0.  
bool(<expr>): Converte para booleano. Numéricos: 0 ou 0.0 → false, qualquer outro valor → true.  
str(<expr>): Converte o valor para sua representação textual.
```

Exemplo:

```
console.log("Digite um número ");  
input(a);  
console.log("Digite três números ");  
input(b, c, d);  
console.log("Soma: ", a+b+c+d);  
console.log();  
console.log(int(3.9));  
console.log(int(true));  
console.log(real(10));  
console.log(real(true));  
console.log(bool(1));  
console.log(bool(0.0));  
console.log(str(10 + 5));  
console.log(str(true));
```

O exemplo acima tem a saída:

```
Digite um número
1
Digite três números
2 3 4
Soma: 10

3
1
10.0
1.0
true
false
15
true
```

4.6 Comandos de controle

O compilador deve reconhecer os seguintes comandos: if-else, while e for conforme as sintaxes abaixo:

- if-else

```
if (<condição >) {
    ...
} ( else if (<condição >) {
    ...
})* [ else {
    ...
}]
```

- while

```
while (<condição >) {
    ...
    [break ]
    ...
}
```

- for

```
for ([< expressão >]; [<condição >]; [<expressão >]) {
    ...
    [break ]
    ...
}
```

4.7 Programa

Um programa é composto por uma função main sem parâmetros e zero ou mais declarações de variáveis, constantes e funções. Todo programa da linguagem Java Script Simplificado deve ser escrito em um arquivo de texto sem formatação com a extensão “.jss”. São permitido comentários apenas de linha usando barras duplas “//”.

A declaração da função main no programa Java Script Simplificado é facultativo..

Exemplo:

```

function int fatorial (int fat) {
    if (fat > 1) {
        console.log(fat);
        return fat * fatorial (fat - 1);
    } else {
        return 1;
    }
}

function void printMedia (int v1 , int v2) {
    const real x = media (v1 ,v2);
    console.log (" Resultado : ", x);
}

function real media ( float n1 , float n2) {
    return (n1 + n2)/2;
}

function void main() { // facultativo a declaração de uma função main
    let int numero ;
    let int n1 , n2;
    console.log (" Programa Fatorial. Digite o valor: ");
    input( numero );
    console.log ( fatorial ( numero ));
    console.log (" Programa Media . Digite os valores: ");
    input(n1 , n2);
    console.log { console.log(Media (n1 , n2));
}

```